

Lab 15 - ASP.NET MVC, Dependency Injection & Middleware

Name: Saif EldenKhaledNazmy Lotfy | LabID:31

Assignment: Lab 15 | ITI Summer Training - Web Development Using .NET

Fingerprinted values for Lab ID 31

Part / Variable	Formula / Derivation	Result
Build Number	$(\text{Lab ID} * 7) + 100 = (31 * 7) + 100$	317
Your Lifetime	$\text{Lab ID} \bmod 3 = 31 \bmod 3$ (0=Trans, 1=Scope, 2=Single)	1 (Scoped)
Audit Path	"/audit-" + Lab ID	/audit-31

Part A - Scaffold the Web Project

Created a new ASP.NET Core MVC project **Lab15_StudentPortalWeb** targeting .NET 10. Rewrote the top-level Program.cs statements into an explicit Program class with a Main method. -

Default Url: **https://localhost:7037**

- Configuration File: **Properties/launchSettings.json**



Part B — Predict Before You Run

B.1 — AddControllersWithViews() added AFTER builder.Build()

Prediction: Compiles fine, but throws at runtime. WebApplicationBuilder freezes its internal IServiceCollection the moment Build() runs; any later builder.Services.Add... call throws an InvalidOperationException ("the service collection cannot be modified...").

Reason: var app = builder.Build() is the dividing line for a reason — it's the exact moment the DI container gets compiled. Registering after that point isn't a smaller mistake, it's a contradiction: there's no container left to add to.

B.2 — A constructor asks for a type nobody registered

Prediction: Not at compile time, not at app startup either — the failure happens the first time a request activates that controller. ASP.NET Core does not walk every controller's constructor at startup by default.

Reason: Confirmed in Part D.2 — commenting out AddDbContext did not stop the app from starting; it only threw once HomeController was instantiated to serve a request.

B.3 — Custom middleware registered AFTER app.UseStaticFiles()

Prediction: The middleware runs once, not twice — only for the HTML page. UseStaticFiles() short-circuits: when it successfully serves a file, it never calls next(), so anything registered after it never sees that request.

Reason: Confirmed in Part F.6-F.7 — moving the middleware after UseStaticFiles() made every .css/.js line disappear from the log; only / remained.

Part C - Wire the Real Context Through DI

Copied entityclasses(Student, Course, Instructor) and StudentPortalContext. Added Constructor with DbContextOptions, deleted OnConfiguring, and registered context via AddDbContext in Program.cs. Injected the context into HomeController and rendered student list.

- Build Heading: **Student Portal - Build #317**
- Students Displayed: **4 students (matches SELECT COUNT(*) FROM Students in SSMS)**

Lab15_StudentPortalWeb Home Privacy

Student Portal — Build #317

Full Name	Year of Study	GPA
Kareem Fouad	4	3.20
Nada Samir	1	3.30
Omar Hesham	3	2.80
Yara Adel	2	3.50

133 % No issues found

Results Messages

	Id	FullName	YearOfStudy	Gpa
1	1	Yara Adel	2	3.5
2	2	Omar Hesham	3	2.8
3	3	Nada Samir	1	3.3
4	4	Kareem Fouad	4	3.2

Part D - Break It On Purpose

D.2 Commented-Out AddDbContext Exception:

```
System.InvalidOperationException: Unable to resolve service for type
'Lab15_StudentPortalWeb.Models.StudentPortalContext' while attempting to activate
'Lab15_StudentPortalWeb.Controllers.HomeController'.
```

- Analysis: The failure occurs only at request time (page load), confirming B.2.

D.5 DbContext Registered as a Singleton:

- Behavior: The application starts up successfully and serves the home page without errors.

D.6 Danger of Singleton DbContext (Reflection):

This is bad news. DbContext tracking state is not thread-safe. Registering it as a Singleton will cause concurrent requests under production load to share the same instance, leading to database state corruption and throwing concurrent access exceptions. This issue is invisible in single-developer testing but fatal in production.

Part E - The Lifetime Experiment (Lab ID: 31)

Created interface `ISaifStampService` and class `SaifStampService` returning the name 'Saif Elden Khaled Nazmy Lotfy' and a random 8-character Guid stamp.

Registered the service in Program.cs with the assigned lifetime: **Scoped (31 mod 3 = 1)**

Request / Refresh	Stamp A (Injected 1)	Stamp B (Injected 2)
First Load	3e353823	3e353823
Second Load (Refresh)	a9ef2808	a9ef2808

Lifetime Analysis and Neighbour Comparison:

- 1. **Within a single load:** Stamp A and Stamp B matched exactly. Since Scoped services create a single instance per request scope, both injections within the same HTTP request resolved to the same instance.
- 2. **Between page loads:** The stamps changed. A page refresh starts a brand new HTTP request, terminating the previous request scope and establishing a new one, causing the container to instantiate a new Scoped service.
- 3. **Neighbour Comparisons:**
 - **(Transient):** Stamp A and B did NOT match in a single load, as Transient instantiates a new object for every injection request. Refreshing the page creates another set of mismatched stamps.
 - **(Singleton):** Stamp A and B matched in a single load, and remained completely identical across all refreshes. The Singleton object is created once at startup and shared globally.

Lab15_StudentPortalWeb

Home

Privacy

Owner: Saif Elden Khaled Nazmy Lotfy | Stamp A: 3e353823 | Stamp B: 3e353823

Student Portal — Build #317

Full Name	Year of Study	GPA
Kareem Fouad	4	3.20
Nada Samir	1	3.30
Omar Hesham	3	2.80
Yara Adel	2	3.50

ib15_StudentPortalWeb

Home

Privacy

Owner: Saif Elden Khaled Nazmy Lotfy | Stamp A: a9ef2808 | Stamp B: a9ef2808

Student Portal — Build #317

Full Name	Year of Study	GPA
Kareem Fouad	4	3.20
Nada Samir	1	3.30
Omar Hesham	3	2.80
Yara Adel	2	3.50

Part F - The Pipeline, Observed

F.4BrowserPageLoadLogs(FirstRun - Middleware at Front):

```
[START] Request path: /  
[END] Request path: /  
[START] Request path: /lib/bootstrap/dist/css/bootstrap.min.css  
[START] Request path: /css/site.css  
[END] Request path: /css/site.css  
[START] Request path: /lib/jquery/dist/jquery.min.js  
[END] Request path: /lib/bootstrap/dist/css/bootstrap.min.css  
[END] Request path: /lib/jquery/dist/jquery.min.js  
[START] Request path: /lib/bootstrap/dist/js/bootstrap.bundle.min.js  
[END] Request path: /lib/bootstrap/dist/js/bootstrap.bundle.min.js  
[START] Request path: /js/site.js  
[END] Request path: /js/site.js  
[START] Request path: /favicon.ico  
[END] Request path: /favicon.ico
```

* **Analysis:** Loading one page triggers multiple requests because the browser parses the HTML document and issues separate, subsequent requests for referenced stylesheets, script files, and icons.

F.5 Audit Path Request Logs (/audit-31):

```
[START] Request path: /audit-31  
[AUDIT] Saif Elden Khaled Nazmy Lotfy saw a request for /audit-31  
[END] Request path: /audit-31
```

* **Explanation:** The middleware pipeline executes for all incoming HTTP requests before routing takes place. Even though `/audit-31` eventually returns a 404, it still flows through the pipeline and triggers our logging.

F.7 Second Run Logs (Middleware after UseStaticFiles):

```
[START] Request path: /  
[END] Request path: /
```

* **Analysis:** Moving the custom middleware after `UseStaticFiles()` causes all static asset requests in `wwwroot` to be short-circuited and served by the static files middleware, never reaching the custom logging code.

Part G - Reflection

1. **DIP Benefit:** Deleting `OnConfiguring` decoupled database configuration from the context itself. Options are passed from outside via DI, allowing the context class to be reusable and redirectable to test or production databases without modifying the context class code.

2. **Fail-Fast:** Startup errors fail immediately when the server boots, alerting developers before deployment. Request-time errors act as silent landmines that might pass automated tests and fail only when a specific user calls a route.

3. **Silent Failures:** (1) Session 13: Multiple Enumeration Bug, executing database queries multiple times silently. (2) Session 14: AsNoTracking change drops, making SaveChangesAsync do nothing silently.

4. **Competing Migrations:** Having two projects manage migration histories causes divergent, conflicting migration histories that lock databases, collide on tables, and corrupt database schema tracking.